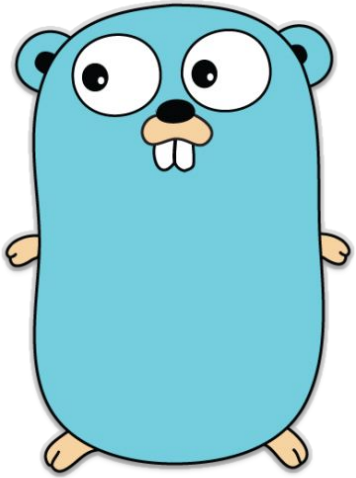
GDG Toledo

Necesito una CLI
¿la escribo en Node o en Golang?

VS



Nivel de madurez

nodejs

Los cambios que en los últimos tiempos experimenta node acerca de cómo se posiciona en cuanto al **tipo de módulos que utiliza *cjs*** está añadiendo mucha incertidumbre. Estado de los módulos nativos de ES6 en nodejs

Ha tenido muchos problemas con forks dividiendo el proyecto y volviendolo a mergear (remember iojs.org), lo cual produjo muchos cambios en el API.

Añadiendo typescript o flow puedes usar strong types e interfaces.

Ecosistema nvm y nodist para desplegar con la versión de node que elijamos

Nivel de madurez

go

Aparece en 2009 y va por la v1.12. Despacito y buena letra :)

Muy orientado al server-side y a operaciones de bajo nivel (concurrencia)

Docker y Kubernetes están escritos en Go ← *cool!*

Statically typed (Strong typed, type inference)

Módulos: hasta la 1.12, había que trabajar en $GOPATH. Con los módulos es posible linkar directorios externos.

Performance

nodejs No hay lugar a dudas en este apartado, node tiene mucha peor performance a la hora de construir este tipo de herramientas. Al final corre sobre su “máquina virtual v8”

go Es muy rápido, puesto que utiliza binarios específicos de la plataforma donde correrán, con todas sus dependencias enlazadas.

Concurrencia

nodejs Node.js es Single-threaded y utiliza un único núcleo del CPU. Cuando se trata de tratar concurrencia en un servidor multi-core, hay algunos acciones que se pueden hacer con Node en la forma de un módulo cluster http://nodejs.org/api/cluster.html. También puedes ejecutar varias instancias del servidor Node.js bastante fácil detrás de un proxy inverso a través de nginx.

go GoRoutines (threads ligeros) y Channels (permiten enviar y recibir información)

Escalabilidad

nodejs Node.js nunca fue creado para resolver el problema de escalado de computación. Fue creado para resolver el problema de escalado de E/S, lo que lo hace muy bien.

go El uso de Goroutines y channels fomenta el paralelismo, por tanto su escalabilidad es superior.

Gestión de Errores

nodejs Hay una gestión clásica de errores mediante try/catch, throw error, tenemos además la primitive error para personalizar new error messages, etc

go La gestión de errores es bastante mejorable. Aunque hace una gestión explícita, no existe un try/catch, sino que se comprueba: *if err == nil {...}*.

Curva de aprendizaje

nodejs Es bastante más sencillo que go para empezar, no hay conceptos como punteros o lenguaje estático a menos que añadas con typescript esta característica. Si se viene de un sistema orientado a eventos es muy sencillo entender el **Event Loop**, manejo asíncrono, async/await, promises etc...

go Viniendo de lenguajes de tipado estático (java, c) es muy fácil. Se pueden utilizar punteros (al contrario que en java) y los parámetros de método se pasan siempre por valor.

CLI’s

¿Qué es una CLI?

CLI: Command Line Interface

Un cliente envía comandos en forma de líneas de texto

Los comandos son enviados en unos pocos caracteres por línea

Permite automatizar tareas repetitivas de una manera muy simple

Aunque usuarios acostumbrados a menús desplegables en una GUI puede que les cueste familiarizarse con ellas

Comandos, subcomandos y flags

Una CLI tiene comandos

```
$ cli remove
```

Cada comando podría tener subcomandos

```
$ cli create task
```

Y cada comando y subcamando podróia tener uno o más flags

```
$ cli create task -d “Kill Cersei Lannister” -t “Season 9”
```

Comandos, subcomandos y flags

En general, ¿cómo es posible comprobar la interfaz o contrato de una CLI?

Comandos, subcomandos y flags

A command and a subcommand debería tener un comando o una flag que informe al usuario de:

- el propósito del comando
- la manera de utilización del comando
- la disponibilidad de subcomandos, si existen
- y la disponibilidad de flags, si existen

```
lpn help
A Fast and Flexible CLI for managing Liferay Portal's Docker images
                built with love by mdelapenya and friends in Go.

Usage:
  lpn [flags]
  lpn [command]

Available Commands:
  checkContainer Checks if there is a container created by lpn
  checkImage     Checks if the proper Liferay Portal image has been pulled by lpn
  deploy         Deploys files or a directory to Liferay Portal's deploy folder in the container run by lpn
  help           Help about any command
  log            Displays logs for the Liferay Portal instance
  pull           Pulls a Liferay Portal Docker image
  rm             Removes the Liferay Portal nook instance
  run            Runs a Liferay Portal instance
  stop           Stops the Liferay Portal nook instance
  update         Updates lpn to the latest version
  version        Print the version number of lpn

Flags:
  -h, --help   help for lpn

Use "lpn [command] --help" for more information about a command.
```

```
$ lpn run nightly -h
Runs a Liferay Portal instance, obtained from Nightly Builds repository: mdelapenya/liferay-portal-nightlies.
    If no image tag is passed to the command, the tag representing the current date [20180413] will be used.

Usage:
  lpn run nightly [flags]

Flags:
  -d, --debug           Enables debug mode. (default false)
  -D, --debugPort int   Sets the debug port of Liferay Portal's bundle. It only applies if debug mode is
                        enabled (default 9000)
  -h, --help            help for nightly
  -p, --httpPort int    Sets the HTTP port of Liferay Portal's bundle. (default 8080)
  -t, --tag string      Sets the image tag to run (default "20180413")
```

Herramientas

Parsear argumentos y opciones

https://github.com/tj/commander.js

https://github.com/yargs/yargs

https://github.com/substack/minimist

Colorear mensajes de información

https://github.com/marak/colors.js/

https://github.com/chalk/chalk

GDG Toledo

Herramientas

Parsear argumentos y opciones, incluso leyendo de configuraciones

https://github.com/spf13/cobra

https://github.com/spf13/viper

Colorear mensajes de información

https://github.com/fatih/color

GDG Toledo

Demo

https://github.com/gdgtoledo/nodeGoCLI

¡Gracias!

GDG Toledo